

Pocket Ledger

Personal expense tracker: complete front-end and back-end source code

7 files · 1310 lines · Repository: [mohan196/git-test](#) · Generated 29 Sep 2026

Technology stack

Front end	HTML5, hand-written CSS (custom properties, Grid, Flexbox, media queries) and plain JavaScript. No framework.
Charts	Chart.js 4.4.1, loaded from cdnjs.cloudflare.com : stacked bar, donut, line and bar charts with tooltips.
Fonts	Google Fonts: Bricolage Grotesque (headings), IBM Plex Sans (text), IBM Plex Mono (numbers).
Back end	Node.js 22 + Express 5 REST API (backend/). Validates input, serves the dashboard, CSV export, optional password login.
Database	SQLite via Node's built-in <code>node:sqlite</code> : tables expenses, budgets, settings. Opened as a plain file, the page uses browser <code>localStorage</code> instead.

Features

- Summary cards: spent this month (vs. last month), budget remaining with status, daily average with projection, top category
- Six-month stacked bar chart of spending by category
- Donut chart of the month's category split, with amounts and percentages
- Running total vs. budget pace, compared with the previous month
- Per-category budgets with editable limits and On track / Near limit / Over budget labels
- Spending by day of week
- EMI & SIP panel: EMIs paid, SIPs invested, their share of outflow and 6-month SIP total
- Add-expense form; transactions table with category filter, search and delete
- Month and currency selectors (INR, USD, EUR, GBP); light and dark themes; phone layout

Expense categories

 Housing	 Food & Dining	 Transport	 Utilities
 Shopping	 Health	 Entertainment	 Other
 SIP	 EMI		

Code layout

Front end: `index.html`

Lines 1–11	Document head: fonts and Chart.js
Lines 12–211	CSS: theme, layout and components
Lines 214–348	HTML: page structure
Lines 349–790	JavaScript: data, charts and interactions

Back end

<code>backend/package.json</code>	Dependencies and npm scripts · 19 lines
<code>backend/src/categories.js</code>	Category ids, default budgets, currencies · 17 lines
<code>backend/src/db.js</code>	SQLite schema and data access · 91 lines
<code>backend/src/server.js</code>	Express REST API, validation, login · 222 lines
<code>backend/test/api.test.js</code>	Automated API tests · 156 lines
<code>Dockerfile</code>	Container image for deployment · 12 lines

Run: `cd backend, npm install, npm start`, then open `http://localhost:3000`. Or open `index.html` directly to use it without a server.

Document head: fonts and Chart.js

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
6 <!-- ARTIFACT-START -->
7 <title>Pocket Ledger</title>
8 <link rel="preconnect" href="https://fonts.googleapis.com">
9 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12..96,
  500;12..96,700&family=IBM+Plex+Sans:wght@400;500;600&family=IBM+Plex+Mono:wght@400;500&display=swap">
11 <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.umd.min.js"></script>
```

CSS: theme, layout and components

```
12 <style>
13 :root {
14   --bg: #f3f5f2;
15   --surface: #fcfcfb;
16   --surface-2: #eefled;
17   --ink: #121715;
18   --ink-2: #4c5652;
19   --muted: #7a8480;
20   --line: #dfe4df;
21   --grid: #e9ece8;
22   --accent: #1d4d45;
23   --accent-ink: #ffffff;
24   --good: #0ca30c; --good-ink: #0a6b0a;
25   --warn: #fab219; --warn-ink: #8a5a00;
26   --bad: #d03b3b; --bad-ink: #b02a2a;
27   --c1: #2a78d6; --c2: #eb6834; --c3: #1baf7a; --c4: #eda100;
28   --c5: #e87ba4; --c6: #008300; --c7: #4a3aa7; --c8: #e34948;
29   --c9: #0091b5; --c10: #9c5a0c;
30   --ghost: #b8c0bc;
31   --display: "Bricolage Grotesque", "Segoe UI", system-ui, sans-serif;
32   --sans: "IBM Plex Sans", "Segoe UI", system-ui, sans-serif;
33   --mono: "IBM Plex Mono", ui-monospace, "SFMono-Regular", Menlo, monospace;
34 }
35 @media (prefers-color-scheme: dark) {
36   :root:not([data-theme="light"]) {
37     color-scheme: dark;
38     --bg: #101312; --surface: #1a1d1c; --surface-2: #232826;
39     --ink: #f1f4f2; --ink-2: #bcc5c1; --muted: #8b9591;
40     --line: #2e3432; --grid: #262b29;
41     --accent: #7fcdcd; --accent-ink: #0d1f1b;
42     --good-ink: #5fd35f; --warn-ink: #fab219; --bad-ink: #f07a7a;
43     --c1: #3987e5; --c2: #d95926; --c3: #199e70; --c4: #c98500;
44     --c5: #d55181; --c6: #008300; --c7: #9085e9; --c8: #e66767;
45     --c9: #1a9cc4; --c10: #b0712a;
46     --ghost: #4d5552;
47   }
48 }
49 :root[data-theme="dark"] {
50   color-scheme: dark;
51   --bg: #101312; --surface: #1a1d1c; --surface-2: #232826;
52   --ink: #f1f4f2; --ink-2: #bcc5c1; --muted: #8b9591;
53   --line: #2e3432; --grid: #262b29;
54   --accent: #7fcdcd; --accent-ink: #0d1f1b;
55   --good-ink: #5fd35f; --warn-ink: #fab219; --bad-ink: #f07a7a;
56   --c1: #3987e5; --c2: #d95926; --c3: #199e70; --c4: #c98500;
57   --c5: #d55181; --c6: #008300; --c7: #9085e9; --c8: #e66767;
58   --c9: #1a9cc4; --c10: #b0712a;
59   --ghost: #4d5552;
60 }
61
62 * { box-sizing: border-box; }
63 html, body { margin: 0; }
64 body {
65   background: var(--bg);
66   color: var(--ink);
67   font: 400 14px/1.5 var(--sans);
68   padding-inline: 16px;
69   padding-block: 20px 48px;
70 }
71 .wrap { max-width: 1280px; margin: 0 auto; display: grid; gap: 16px; }
72 .num { font-family: var(--mono); font-variant-numeric: tabular-nums; }
73
```

```

74  /* Header */
75  header.top { display: flex; flex-wrap: wrap; align-items: end; justify-content: space-between; gap: 16px;
↳ padding-block: 4px 8px; }
76  .brand h1 { font: 700 clamp(26px, 4vw, 36px)/1.05 var(--display); margin: 0; letter-spacing: -0.02em; tex
↳ t-wrap: balance; }
77  .brand p { margin: 4px 0 0; color: var(--ink-2); }
78  .controls { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; }
79  label.inline { display: flex; flex-direction: column; gap: 2px; font-size: 11px; text-transform: uppercase
↳ e; letter-spacing: 0.06em; color: var(--muted); }
80  select, input, button { font: inherit; color: inherit; }
81  select, input {
82    background: var(--surface); border: 1px solid var(--line); border-radius: 8px;
83    padding: 7px 10px; min-height: 36px; color: var(--ink);
84  }
85  select:focus-visible, input:focus-visible, button:focus-visible { outline: 2px solid var(--accent); outli
↳ ne-offset: 2px; }
86  .btn {
87    border: 1px solid var(--line); background: var(--surface); color: var(--ink);
88    border-radius: 8px; padding: 7px 14px; min-height: 36px; cursor: pointer; font-weight: 500;
89  }
90  .btn:hover { background: var(--surface-2); }
91  .btn.primary { background: var(--accent); color: var(--accent-ink); border-color: var(--accent); }
92  .btn.primary:hover { filter: brightness(1.08); }
93  .btn.danger-armed { border-color: var(--bad); color: var(--bad-ink); }
94
95  .notice {
96    display: flex; flex-wrap: wrap; gap: 8px 16px; align-items: center; justify-content: space-between;
97    background: var(--surface-2); border: 1px dashed var(--line); border-radius: 10px; padding: 10px 14px;
↳ color: var(--ink-2);
98  }
99  .notice.strong { color: var(--ink); }
100 .notice.error { border: 1px solid var(--bad); color: var(--bad-ink); border-style: solid; }
101 .store { display: inline-flex; align-items: center; gap: 5px; margin-left: 6px; font-size: 12px; color: v
↳ ar(--muted); white-space: nowrap; }
102 .store::before { content: ""; width: 7px; height: 7px; border-radius: 50%; background: var(--muted); }
103 .store.server::before { background: var(--good); }
104 .foot a { color: var(--accent); }
105
106 /* KPI tiles */
107 .kpis { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 16px; }
108 .kpi { background: var(--surface); border: 1px solid var(--line); border-radius: 12px; padding: 16px 18px
↳ ; display: grid; gap: 6px; align-content: start; }
109 .kpi .label { font-size: 11px; text-transform: uppercase; letter-spacing: 0.07em; color: var(--muted); fo
↳ nt-weight: 600; }
110 .kpi .value { font: 500 clamp(22px, 2.6vw, 30px)/1.1 var(--mono); font-variant-numeric: tabular-nums; let
↳ ter-spacing: -0.02em; }
111 .kpi .sub { color: var(--ink-2); font-size: 13px; display: flex; gap: 6px; align-items: center; flex-wrap
↳ : wrap; }
112 .meter { height: 6px; border-radius: 3px; background: var(--surface-2); overflow: hidden; }
113 .meter > span { display: block; height: 100%; border-radius: 3px; background: var(--accent); }
114
115 .pill { display: inline-flex; align-items: center; gap: 5px; padding: 1px 8px 1px 6px; border-radius: 999
↳ px; font-size: 12px; font-weight: 600; border: 1px solid currentColor; line-height: 1.6; white-space: nowra
↳ p; }
116 .pill::before { content: ""; width: 7px; height: 7px; border-radius: 50%; background: var(--dot); }
117 .pill.good { color: var(--good-ink); --dot: var(--good); }
118 .pill.warn { color: var(--warn-ink); --dot: var(--warn); }
119 .pill.bad { color: var(--bad-ink); --dot: var(--bad); }
120 .delta.up { color: var(--bad-ink); } .delta.down { color: var(--good-ink); }
121
122 /* Panels */
123 .grid { display: grid; grid-template-columns: repeat(12, minmax(0, 1fr)); gap: 16px; }
124 .panel { background: var(--surface); border: 1px solid var(--line); border-radius: 12px; padding: 16px 18
↳ px; display: grid; gap: 12px; align-content: start; min-width: 0; }
125 .panel h2 { font: 600 16px/1.3 var(--display); margin: 0; letter-spacing: -0.01em; }
126 .panel .head { display: flex; justify-content: space-between; align-items: baseline; gap: 12px; flex-wrap
↳ : wrap; }
127 .panel .hint { color: var(--muted); font-size: 12px; }
128 .span-8 { grid-column: span 8; } .span-7 { grid-column: span 7; } .span-5 { grid-column: span 5; } .span-
↳ 4 { grid-column: span 4; }
129 .stack { display: grid; gap: 16px; align-content: start; min-width: 0; }
130 .chart-box { position: relative; height: 280px; }
131 .chart-box.short { height: 200px; }
132 .chart-box.tall { height: 400px; }
133 .chart-box.mid { height: 340px; }
134
135 .legend { display: flex; flex-wrap: wrap; gap: 6px 14px; font-size: 12px; color: var(--ink-2); }
136 .legend span { display: inline-flex; align-items: center; gap: 6px; }
137 .swatch { width: 10px; height: 10px; border-radius: 3px; background: var(--c); flex: none; }

```

```

138 .swatch.line { height: 2px; border-radius: 1px; width: 14px; }
139 .swatch.dash { height: 0; width: 14px; border-top: 2px dashed var(--c); background: none; }
140
141 /* Donut */
142 .donut-wrap { position: relative; height: 220px; }
143 .donut-center { position: absolute; inset: 0; display: grid; place-content: center; text-align: center; pointer-events: none; }
144 .donut-center .v { font: 500 20px/1.1 var(--mono); font-variant-numeric: tabular-nums; }
145 .donut-center .l { font-size: 11px; color: var(--muted); text-transform: uppercase; letter-spacing: 0.07em; }
146 .cat-list { display: grid; gap: 2px; margin: 0; padding: 0; list-style: none; }
147 .cat-list li { display: grid; grid-template-columns: auto 1fr auto auto; gap: 10px; align-items: center; padding: 5px 4px; border-radius: 6px; }
148 .cat-list li:hover { background: var(--surface-2); }
149 .cat-list .amt { font-family: var(--mono); font-variant-numeric: tabular-nums; }
150 .cat-list .pct { font-family: var(--mono); font-variant-numeric: tabular-nums; color: var(--muted); width: 3.5em; text-align: right; }
151
152 /* Budgets */
153 .budgets { display: grid; gap: 12px; margin: 0; padding: 0; list-style: none; }
154 .budgets li { display: grid; gap: 5px; }
155 .budgets .row { display: flex; justify-content: space-between; align-items: center; gap: 8px; flex-wrap: wrap; }
156 .budgets .name { display: inline-flex; align-items: center; gap: 8px; font-weight: 500; }
157 .budgets .fig { font-family: var(--mono); font-variant-numeric: tabular-nums; font-size: 13px; color: var(--ink-2); display: inline-flex; align-items: center; gap: 4px; }
158 .budgets input { width: 92px; min-height: 28px; padding: 2px 6px; font-family: var(--mono); font-size: 13px; text-align: right; }
159 .bar { position: relative; height: 8px; border-radius: 4px; background: var(--surface-2); overflow: hidden; }
160 .bar > span { position: absolute; inset: 0 auto 0 0; border-radius: 4px; background: var(--c); }
161 .bar.over > span { background: repeating-linear-gradient(135deg, var(--c) 0 6px, color-mix(in srgb, var(--c) 60%, var(--surface)) 6px 9px); }
162
163 /* EMI & SIP */
164 .commit-tiles { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
165 .commit { background: var(--surface-2); border-radius: 10px; padding: 10px 12px; display: grid; gap: 2px; }
166 .commit .label { display: inline-flex; align-items: center; gap: 6px; font-size: 11px; text-transform: uppercase; letter-spacing: 0.06em; color: var(--muted); font-weight: 600; }
167 .commit .v { font-size: 18px; font-weight: 500; }
168 .commit .s { font-size: 12px; color: var(--ink-2); }
169 .commit-note { margin: 0; font-size: 13px; color: var(--ink-2); }
170 .commit-list { list-style: none; margin: 0; padding: 0; display: grid; gap: 2px; }
171 .commit-list li { display: grid; grid-template-columns: auto 1fr auto; gap: 8px; align-items: center; padding: 4px 2px; border-bottom: 1px solid var(--grid); font-size: 13px; }
172 .commit-list li:last-child { border-bottom: 0; }
173 .commit-list .d { color: var(--muted); font-size: 12px; }
174 .commit-list .a { font-family: var(--mono); font-variant-numeric: tabular-nums; }
175
176 /* Form */
177 form.add { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
178 form.add .full { grid-column: 1 / -1; }
179 form.add label { display: grid; gap: 4px; font-size: 12px; color: var(--ink-2); font-weight: 500; }
180 form.add input, form.add select { width: 100%; }
181 .form-msg { font-size: 13px; min-height: 1.5em; color: var(--ink-2); }
182 .form-msg.err { color: var(--bad-ink); }
183
184 /* Transactions */
185 .filters { display: flex; flex-wrap: wrap; gap: 6px; }
186 .chip { display: inline-flex; align-items: center; gap: 6px; border: 1px solid var(--line); background: var(--surface); color: var(--ink-2); border-radius: 999px; padding: 3px 10px; font-size: 12px; cursor: pointer; min-height: 28px; }
187 .chip[aria-pressed="true"] { background: var(--ink); color: var(--bg); border-color: var(--ink); }
188 .chip .swatch { width: 8px; height: 8px; border-radius: 50%; }
189 .table-scroll { overflow-x: auto; max-height: 460px; overflow-y: auto; border-top: 1px solid var(--line); }
190 table { width: 100%; border-collapse: collapse; font-size: 13px; }
191 th, td { text-align: left; padding: 8px 6px; border-bottom: 1px solid var(--grid); white-space: nowrap; }
192 th { position: sticky; top: 0; background: var(--surface); font-size: 11px; text-transform: uppercase; letter-spacing: 0.06em; color: var(--muted); font-weight: 600; }
193 td.amt, th.amt { text-align: right; font-family: var(--mono); font-variant-numeric: tabular-nums; }
194 td.note { white-space: normal; min-width: 140px; color: var(--ink-2); }
195 .cat-tag { display: inline-flex; align-items: center; gap: 6px; }
196 .del { border: 0; background: none; color: var(--muted); cursor: pointer; font-size: 16px; line-height: 1; padding: 4px 8px; border-radius: 6px; }
197 .del:hover { color: var(--bad-ink); background: var(--surface-2); }
198 .empty { color: var(--muted); padding: 16px 4px; text-align: center; }
199 .foot { display: flex; justify-content: space-between; gap: 8px; flex-wrap: wrap; color: var(--ink-2); font-

```

```

    nt-size: 13px; }
200
201 @media (max-width: 1000px) {
202     .span-8, .span-7, .span-5, .span-4 { grid-column: 1 / -1; }
203     .kpi { grid-template-columns: repeat(2, minmax(0, 1fr)); }
204 }
205 @media (max-width: 460px) {
206     .kpi { grid-template-columns: 1fr; }
207     form.add { grid-template-columns: 1fr; }
208     .chart-box, .chart-box.tall, .chart-box.mid { height: 260px; }
209 }
210 @media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none !important; }
    }
211 </style>
212 <!-- ARTIFACT-HEAD-END -->
213 </head>

```

HTML: page structure

```

214 <body>
215 <!-- ARTIFACT-BODY-START -->
216 <div class="wrap">
217     <header class="top">
218         <div class="brand">
219             <h1>Pocket Ledger</h1>
220             <p>Where your money went, by month and by category. <span class="store" id="storeLabel">Saved in this
    browser</span></p>
221         </div>
222         <div class="controls">
223             <label class="inline" for="monthSel">Month
224                 <select id="monthSel"></select>
225             </label>
226             <label class="inline" for="curSel">Currency
227                 <select id="curSel">
228                     <option value="INR">₹ INR</option>
229                     <option value="USD">$ USD</option>
230                     <option value="EUR">€ EUR</option>
231                     <option value="GBP">£ GBP</option>
232                 </select>
233             </label>
234         </div>
235     </header>
236
237     <div class="notice error" id="syncMsg" role="alert" hidden></div>
238
239     <div class="notice" id="sampleNotice" hidden>
240         <span><strong>You're looking at sample data.</strong> Add your own expenses below, or clear the sample
    to start fresh. <span id="storeText">Everything is saved in this browser only.</span></span>
241         <button class="btn" id="clearBtn" type="button">Clear sample data</button>
242     </div>
243
244     <section class="kpi" aria-label="Month summary">
245         <div class="kpi">
246             <span class="label">Spent this month</span>
247             <span class="value" id="kSpent"></span>
248             <span class="sub" id="kSpentSub"></span>
249         </div>
250         <div class="kpi">
251             <span class="label">Budget remaining</span>
252             <span class="value" id="kLeft"></span>
253             <div class="meter" aria-hidden="true"><span id="kMeter" style="width:0%"></span></div>
254             <span class="sub" id="kLeftSub"></span>
255         </div>
256         <div class="kpi">
257             <span class="label">Daily average</span>
258             <span class="value" id="kDaily"></span>
259             <span class="sub" id="kDailySub"></span>
260         </div>
261         <div class="kpi">
262             <span class="label">Top category</span>
263             <span class="value" id="kTop" style="font-family:var(--display);font-weight:600;letter-spacing:-0.01e
    m"></span>
264             <span class="sub" id="kTopSub"></span>
265         </div>
266     </section>
267
268     <section class="grid">
269         <div class="panel span-8">
270             <div class="head"><h2>Six-month spending by category</h2><span class="hint">Hover a bar for the break

```

```

    down</span></div>
271     <div class="legend" id="catLegend"></div>
272     <div class="chart-box tall"><canvas id="trendChart" aria-label="Stacked bar chart of monthly spending
    by category"></canvas></div>
273     </div>
274     <div class="panel span-4">
275         <div class="head"><h2>Where it went</h2><span class="hint" id="donutHint"></span></div>
276         <div class="donut-wrap">
277             <canvas id="donutChart" aria-label="Donut chart of spending share by category"></canvas>
278             <div class="donut-center"><span class="v num" id="donutTotal"></span><span class="l">total</span><
    /div>
279         </div>
280         <ul class="cat-list" id="catList"></ul>
281     </div>
282
283     <div class="panel span-7">
284         <div class="head"><h2>Running total vs. budget pace</h2><span class="hint">Cumulative spend through t
    he month</span></div>
285         <div class="legend">
286             <span><i class="swatch line" style="--c:var(--c1)"></i>This month</span>
287             <span><i class="swatch line" style="--c:var(--ghost)"></i>Previous month</span>
288             <span><i class="swatch dash" style="--c:var(--muted)"></i>Budget pace</span>
289         </div>
290         <div class="chart-box mid"><canvas id="paceChart" aria-label="Line chart of cumulative spending again
    st budget pace"></canvas></div>
291     </div>
292     <div class="panel span-5">
293         <div class="head"><h2>Category budgets</h2><span class="hint">Edit a limit to change it</span></div>
294         <ul class="budgets" id="budgetList"></ul>
295     </div>
296
297     <div class="stack span-4">
298         <div class="panel">
299             <h2>Add an expense</h2>
300             <form class="add" id="addForm" novalidate>
301                 <label for="fAmount">Amount
302                 <input id="fAmount" type="number" inputmode="decimal" min="0" step="0.01" placeholder="0.00" re
    quired>
303                 </label>
304                 <label for="fDate">Date
305                 <input id="fDate" type="date" required>
306                 </label>
307                 <label class="full" for="fCat">Category
308                 <select id="fCat"></select>
309                 </label>
310                 <label class="full" for="fNote">Note
311                 <input id="fNote" type="text" maxlength="80" placeholder="e.g. Weekly groceries">
312                 </label>
313                 <button class="btn primary full" type="submit">Add expense</button>
314                 <div class="form-msg full" id="formMsg" role="status"></div>
315             </form>
316         </div>
317         <div class="panel">
318             <div class="head"><h2>EMI & SIP</h2><span class="hint" id="commitHint"></span></div>
319             <div class="commit-tiles">
320                 <div class="commit"><span class="label"><i class="swatch" style="--c:var(--c10)"></i>EMIs paid</s
    pan><span class="v num" id="emiTotal"></span><span class="s" id="emiSub"></span></div>
321                 <div class="commit"><span class="label"><i class="swatch" style="--c:var(--c9)"></i>SIPs invested
    </span><span class="v num" id="sipTotal"></span><span class="s" id="sipSub"></span></div>
322             </div>
323             <p class="commit-note" id="commitNote"></p>
324             <ul class="commit-list" id="commitList"></ul>
325         </div>
326         <div class="panel">
327             <div class="head"><h2>By day of week</h2><span class="hint">Selected month</span></div>
328             <div class="chart-box short"><canvas id="dowChart" aria-label="Bar chart of spending by day of week
    "></canvas></div>
329         </div>
330     </div>
331
332     <div class="panel span-8">
333         <div class="head">
334             <h2>Transactions</h2>
335             <input id="search" type="search" placeholder="Search notes" aria-label="Search notes" style="min-he
    ight:32px;padding:4px 10px">
336         </div>
337         <div class="filters" id="filters" role="group" aria-label="Filter by category"></div>
338         <div class="table-scroll">
339             <table>

```

```

340         <thead><tr><th>Date</th><th>Category</th><th>Note</th><th class="amt">Amount</th><th><span class=
  ↳ "sr" hidden>Actions</span></th></tr></thead>
341         <tbody id="txBody"></tbody>
342     </table>
343 </div>
344     <div class="foot"><span id="txCount"></span><a id="csvLink" href="api/export.csv" hidden>Download all
  ↳ as CSV</a><span id="txSum" class="num"></span></div>
345 </div>
346 </section>
347 </div>
348

```

JavaScript: data, charts and interactions

```

349 <script>
350 (() => {
351     const CATS = [
352         { id: "housing", name: "Housing", v: "--c1", budget: 18000 },
353         { id: "food", name: "Food & Dining", v: "--c2", budget: 12000 },
354         { id: "transport", name: "Transport", v: "--c3", budget: 6000 },
355         { id: "utilities", name: "Utilities", v: "--c4", budget: 4500 },
356         { id: "shopping", name: "Shopping", v: "--c5", budget: 6000 },
357         { id: "health", name: "Health", v: "--c6", budget: 3000 },
358         { id: "fun", name: "Entertainment", v: "--c7", budget: 3500 },
359         { id: "other", name: "Other", v: "--c8", budget: 2500 },
360         { id: "sip", name: "SIP", v: "--c9", budget: 8000 },
361         { id: "emi", name: "EMI", v: "--c10", budget: 12200 },
362     ];
363     const CAT = Object.fromEntries(CATS.map(c => [c.id, c]));
364     const KEY = "pocket-ledger-v1";
365     const LOCALES = { INR: "en-IN", USD: "en-US", EUR: "en-IE", GBP: "en-GB" };
366
367     const pad = n => String(n).padStart(2, "0");
368     const ymd = d => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
369     const ym = d => `${d.getFullYear()}-${pad(d.getMonth() + 1)}`;
370     const today = new Date();
371     const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 8).padEnd(6, "0");
372
373     // ----- Sample data (deterministic) -----
374     function mulberry32(a) { return () => { a |= 0; a = a + 0x6D2B79F5 | 0; let t = Math.imul(a ^ a >>> 15, 1
  ↳ | a); t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t; return ((t ^ t >>> 14) >>> 0) / 4294967296; }; }
375     function buildSample() {
376         const r = mulberry32(42);
377         const rand = (lo, hi) => Math.round((lo + r() * (hi - lo)) / 10) * 10;
378         const out = [];
379         for (let m = 5; m >= 0; m--) {
380             const first = new Date(today.getFullYear(), today.getMonth() - m, 1);
381             const dim = new Date(first.getFullYear(), first.getMonth() + 1, 0).getDate();
382             const lastDay = m === 0 ? today.getDate() : dim;
383             const add = (day, cat, amount, note) => {
384                 if (day > lastDay) return;
385                 out.push({ id: uid(), date: ymd(new Date(first.getFullYear(), first.getMonth(), day)), cat, amount,
  ↳ note });
386             };
387             const any = () => 1 + Math.floor(r() * dim);
388             add(1, "housing", 18000, "Rent");
389             add(5, "utilities", rand(1600, 2700), "Electricity bill");
390             add(10, "utilities", 799, "Home internet");
391             add(12, "utilities", 399, "Mobile recharge");
392             for (let i = 0; i < 5; i++) add(any(), "food", rand(700, 2300), ["Groceries", "Vegetables & fruit", "
  ↳ Supermarket run", "Milk & bakery"][i % 4]);
393             for (let i = 0, n = 5 + Math.floor(r() * 5); i < n; i++) add(any(), "food", rand(220, 1300), ["Dinner
  ↳ out", "Lunch with team", "Food delivery", "Coffee & snacks", "Weekend brunch"][i % 5]);
394             for (let i = 0, n = 8 + Math.floor(r() * 6); i < n; i++) add(any(), "transport", rand(80, 420), ["Cab
  ↳ ride", "Auto", "Metro card top-up", "Parking"][i % 4]);
395             add(any(), "transport", rand(1500, 2400), "Fuel");
396             if (r() > 0.4) add(any(), "transport", rand(1500, 2400), "Fuel");
397             for (let i = 0, n = 2 + Math.floor(r() * 4); i < n; i++) add(any(), "shopping", rand(450, 3400), ["Cl
  ↳ othes", "Household items", "Online order", "Shoes", "Gift"][i % 5]);
398             for (let i = 0, n = 1 + Math.floor(r() * 2); i < n; i++) add(any(), "health", rand(300, 2400), ["Phar
  ↳ macy", "Doctor visit", "Gym membership"][i % 3]);
399             add(3, "fun", 649, "Streaming subscription");
400             for (let i = 0, n = 1 + Math.floor(r() * 4); i < n; i++) add(any(), "fun", rand(250, 1600), ["Movie t
  ↳ ickets", "Concert", "Games", "Books"][i % 4]);
401             add(5, "emi", 9800, "Car loan EMI");
402             add(15, "emi", 2400, "Phone EMI");
403             add(7, "sip", 5000, "Nifty 50 index fund SIP");
404             add(10, "sip", 3000, "Flexi-cap fund SIP");
405             for (let i = 0, n = 1 + Math.floor(r() * 3); i < n; i++) add(any(), "other", rand(200, 1400), ["Hairc

```

```

    ut", "Donation", "Stationery", "Laundry"][i % 4]);
406     }
407     return out;
408 }
409
410 // ----- State -----
411 // Data lives on the backend (backend/ folder) when the page is served by it.
412 // Opened any other way (as a file, on GitHub Pages), it falls back to this browser's localStorage.
413 const DEFAULT_BUDGETS = Object.fromEntries(CATS.map(c => [c.id, c.budget]));
414 const sampleLedger = () => ({ expenses: buildSample(), budgets: { ...DEFAULT_BUDGETS }, currency: "INR",
    sample: true });
415 function readLocal() {
416     try { const s = JSON.parse(localStorage.getItem(KEY)); if (s && Array.isArray(s.expenses)) return s; }
    catch (e) {}
417     return null;
418 }
419 function saveLocal() { try { localStorage.setItem(KEY, JSON.stringify(state)); } catch (e) {} }
420 const state = readLocal() || sampleLedger();
421 state.budgets = { ...DEFAULT_BUDGETS, ...state.budgets };
422 if (state.sample && !state.expenses.some(e => e.cat === "emi")) { state.expenses = buildSample(); saveLocal(); }
423
424 let serverMode = false;
425 const toApi = e => ({ id: e.id, date: e.date, category: e.cat, amount: e.amount, note: e.note });
426 const fromApi = e => ({ id: e.id, date: e.date, cat: e.category, amount: e.amount, note: e.note });
427 async function api(method, path, body) {
428     const res = await fetch("api/" + path, {
429         method, cache: "no-store",
430         headers: body ? { "Content-Type": "application/json" } : {},
431         body: body ? JSON.stringify(body) : undefined,
432     });
433     if (!res.ok) {
434         let msg = `the server answered ${res.status}`;
435         try { msg = (await res.json()).error || msg; } catch (e) {}
436         throw new Error(msg);
437     }
438     return res.status === 204 ? null : res.json();
439 }
440 async function detectServer() {
441     if (!/^https?:$/.test(location.protocol)) return false;
442     try { const r = await fetch("api/health", { cache: "no-store" }); return r.ok && (await r.json()).ok ==
    = true; }
443     catch (e) { return false; }
444 }
445 async function loadFromServer() {
446     let s = await api("GET", "state");
447     if (!s.initialized) {
448         // First run: move this browser's own ledger to the server if it has one, otherwise start with sample
    data.
449         const local = readLocal();
450         const seed = local && !local.sample ? local : sampleLedger();
451         await api("POST", "import", { expenses: seed.expenses.map(toApi), budgets: { ...DEFAULT_BUDGETS, ...s
    eed.budgets }, currency: seed.currency, sample: !!seed.sample });
452         s = await api("GET", "state");
453     }
454     state.expenses = s.expenses.map(fromApi);
455     state.budgets = { ...DEFAULT_BUDGETS, ...s.budgets };
456     state.currency = s.currency;
457     state.sample = s.sample;
458 }
459 // Saves a change: sends it to the server in server mode, or writes localStorage otherwise.
460 function persist(request) {
461     if (!serverMode) { saveLocal(); return; }
462     request().catch(err => showSyncError(`Couldn't save to the server: ${err.message}. Reload the page to s
    ee what was saved.`));
463 }
464 function showSyncError(text) { $("syncMsg").textContent = text; $("syncMsg").hidden = false; }
465 let month = ym(today);
466 let filterCat = "all";
467 let query = "";
468
469 let fmt, fmtShort;
470 function setCurrency() {
471     const loc = LOCALES[state.currency] || "en-US";
472     fmt = new Intl.NumberFormat(loc, { style: "currency", currency: state.currency, maximumFractionDigits:
    0 });
473     fmtShort = new Intl.NumberFormat("en-US", { style: "currency", currency: state.currency, notation: "com
    pact", maximumFractionDigits: 1 });
474 }

```

```

475 const money = v => fmt.format(v);
476
477 // ----- Helpers -----
478 const $ = id => document.getElementById(id);
479 const css = v => getComputedStyle(document.documentElement).getPropertyValue(v).trim();
480 const color = c => css(CAT[c].v);
481 const inMonth = key => state.expenses.filter(e => e.date.startsWith(key));
482 const sum = arr => arr.reduce((a, e) => a + e.amount, 0);
483 const monthDate = key => { const [y, m] = key.split("-").map(Number); return new Date(y, m - 1, 1); };
484 const shiftMonth = (key, n) => { const d = monthDate(key); return ym(new Date(d.getFullYear(), d.getMonth()
↳ () + n, 1)); };
485 const monthLabel = (key, style = "long") => monthDate(key).toLocaleDateString("en-US", { month: style, ye
↳ ar: "numeric" });
486 const daysIn = key => { const d = monthDate(key); return new Date(d.getFullYear(), d.getMonth() + 1, 0).g
↳ etDate(); };
487 const byCat = list => { const o = Object.fromEntries(CATS.map(c => [c.id, 0])); list.forEach(e => { o[e.c
↳ at] = (o[e.cat] || 0) + e.amount; }); return o; };
488 const esc = s => String(s).replace(/&<"'/g, ch => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&qu
↳ ot;", "\"": "&#39;" }[ch]));
489 const totalBudget = () => CATS.reduce((a, c) => a + (state.budgets[c.id] || 0), 0);
490 const status = pct => pct > 1 ? ["bad", "Over budget"] : pct >= 0.8 ? ["warn", "Near limit"] : ["good", "
↳ On track"];
491
492 // ----- Controls -----
493 function fillMonths() {
494   const keys = new Set(state.expenses.map(e => e.date.slice(0, 7)));
495   keys.add(ym(today));
496   const sorted = [...keys].sort().reverse();
497   if (!sorted.includes(month)) month = sorted[0];
498   $("monthSel").innerHTML = sorted.map(k => `<option value="${k}"${k === month ? " selected" : ""}>${mont
↳ hLabel(k)}</option>`).join("");
499 }
500 $("monthSel").addEventListener("change", e => { month = e.target.value; render(); });
501 $("curSel").value = state.currency;
502 $("curSel").addEventListener("change", e => {
503   state.currency = e.target.value; setCurrency(); render();
504   persist() => api("PUT", "settings", { currency: state.currency });
505 });
506 $("fCat").innerHTML = CATS.map(c => `<option value="${c.id}">${c.name}</option>`).join("");
507 $("fDate").value = ymd(today);
508 $("catLegend").innerHTML = CATS.map(c => `<span><i class="swatch" style="--c:var(${c.v})"></i>${c.name}</
↳ span>`).join("");
509
510 $("addForm").addEventListener("submit", e => {
511   e.preventDefault();
512   const amount = parseFloat($("fAmount").value);
513   const date = $("fDate").value;
514   const msg = $("formMsg");
515   if (!(amount > 0)) { msg.className = "form-msg full err"; msg.textContent = "Enter an amount greater th
↳ an zero."; $("fAmount").focus(); return; }
516   if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(date)) { msg.className = "form-msg full err"; msg.textContent = "Pick a
↳ date for this expense."; $("fDate").focus(); return; }
517   const cat = $("fCat").value;
518   const note = $("fNote").value.trim() || CAT[cat].name;
519   const exp = { id: uid(), date, cat, amount: Math.round(amount * 100) / 100, note };
520   state.expenses.push(exp);
521   persist() => api("POST", "expenses", toApi(exp));
522   month = date.slice(0, 7);
523   msg.className = "form-msg full";
524   msg.textContent = `Added ${money(amount)} to ${CAT[cat].name}.`;
525   $("fAmount").value = ""; $("fNote").value = "";
526   render();
527 });
528
529 let clearArmed = false;
530 $("clearBtn").addEventListener("click", () => {
531   const b = $("clearBtn");
532   if (!clearArmed) { clearArmed = true; b.textContent = "Click again to clear"; b.classList.add("danger-a
↳ rmed"); setTimeout(() => { clearArmed = false; b.textContent = "Clear sample data"; b.classList.remove("dan
↳ ger-armed"); }, 4000); return; }
533   state.expenses = []; state.sample = false; month = ym(today); render();
534   persist() => api("DELETE", "expenses");
535 });
536
537 $("search").addEventListener("input", e => { query = e.target.value.toLowerCase(); renderTable(); });
538
539 // ----- Charts -----
540 const charts = {};
541 function baseOpts() {

```

```

542     const ink2 = css("--ink-2"), grid = css("--grid"), surface = css("--surface"), ink = css("--ink"), line
543     = css("--line");
544     return {
545       responsive: true, maintainAspectRatio: false, animation: { duration: 400 },
546       plugins: {
547         legend: { display: false },
548         tooltip: {
549           backgroundColor: surface, titleColor: ink, bodyColor: ink2, borderColor: line, borderWidth: 1,
550           padding: 10, cornerRadius: 8, boxPadding: 4, usePointStyle: true,
551           titleFont: { family: "IBM Plex Sans", weight: "600" }, bodyFont: { family: "IBM Plex Mono" },
552         },
553         scales: {
554           x: { grid: { display: false }, border: { color: line }, ticks: { color: ink2, font: { family: "IBM
555             Plex Sans", size: 12 } } },
556           y: { beginAtZero: true, grid: { color: grid }, border: { display: false }, ticks: { color: ink2, fo
557             nt: { family: "IBM Plex Mono", size: 11 }, callback: v => fmtShort.format(v), maxTicksLimit: 6 } },
558         },
559       };
560     }
561     function makeChart(id, cfg) { if (charts[id]) charts[id].destroy(); charts[id] = new Chart($(id), cfg); }
562     function renderTrend() {
563       const keys = Array.from({ length: 6 }, (_, i) => shiftMonth(month, i - 5));
564       const surface = css("--surface");
565       const datasets = CATS.map((c, i) => ({
566         label: c.name,
567         data: keys.map(k => byCat(inMonth(k))[c.id]),
568         backgroundColor: color(c.id),
569         borderColor: surface,
570         borderWidth: { top: 2, left: 0, right: 0, bottom: 0 },
571         borderRadius: i === CATS.length - 1 ? { topLeft: 4, topRight: 4 } : 0,
572         borderSkipped: "bottom",
573         maxBarThickness: 44,
574         stack: "s",
575       }));
576       const opts = baseOpts();
577       opts.scales.x.stacked = true; opts.scales.y.stacked = true;
578       opts.interaction = { mode: "index", intersect: false };
579       opts.plugins.tooltip.itemSort = (a, b) => b.raw - a.raw;
580       opts.plugins.tooltip.filter = item => item.raw > 0;
581       opts.plugins.tooltip.callbacks = {
582         label: ctx => ` ${ctx.dataset.label}: ${money(ctx.raw)} `,
583         footer: items => `Total: ${money(items.reduce((a, i) => a + i.raw, 0))}` ,
584       };
585       opts.plugins.tooltip.footerFont = { family: "IBM Plex Mono", weight: "600" };
586       opts.plugins.tooltip.footerColor = css("--ink");
587       makeChart("trendChart", { type: "bar", data: { labels: keys.map(k => monthDate(k).toLocaleDateString("en-US", { month: "short" }) + (k === month ? " ●" : "")), datasets }, options: opts });
588     }
589     function renderDonut(cats, total) {
590       const surface = css("--surface");
591       const order = CATS.filter(c => cats[c.id] > 0);
592       makeChart("donutChart", {
593         type: "doughnut",
594         data: { labels: order.map(c => c.name), datasets: [{ data: order.map(c => cats[c.id]), backgroundColo
595           r: order.map(c => color(c.id)), borderColor: surface, borderWidth: 2, hoverOffset: 6, borderRadius: 3 } ] },
596         options: {
597           responsive: true, maintainAspectRatio: false, cutout: "70%", layout: { padding: 6 },
598           plugins: {
599             legend: { display: false },
600             tooltip: { ...baseOpts().plugins.tooltip, callbacks: { label: ctx => ` ${money(ctx.raw)} · $((ctx
601               .x.raw / total) * 100).toFixed(1)}%` } },
602           },
603         });
604       $("donutTotal").textContent = money(total);
605       $("donutHint").textContent = monthLabel(month, "short");
606       const sorted = [...CATS].sort((a, b) => cats[b.id] - cats[a.id]);
607       $("catList").innerHTML = sorted.map(c => `<li><i class="swatch" style="--c:var(${c.v})"></i><span>${(c.n
608         ame)}</span><span class="amt">${money(cats[c.id])}</span><span class="pct">${total ? Math.round(cats[c.id] /
609         total * 100) : 0}%</span></li>`).join("");
610     }
611     function renderPace() {
612       const dim = daysIn(month);
613       const isCurrent = month === ym(today);
614       const lastDay = isCurrent ? today.getDate() : dim;

```

```

613     const cum = key => {
614         const n = daysIn(key); const daily = Array(n).fill(0);
615         inMonth(key).forEach(e => { daily[Number(e.date.slice(8, 10)) - 1] += e.amount; });
616         let run = 0; return daily.map(v => (run += v));
617     };
618     const cur = cum(month).map((v, i) => (i < lastDay ? v : null));
619     const prevRaw = cum(shiftMonth(month, -1));
620     const prev = Array.from({ length: dim }, (_, i) => prevRaw[Math.min(i, prevRaw.length - 1)]);
621     const tb = totalBudget();
622     const pace = Array.from({ length: dim }, (_, i) => Math.round(tb * (i + 1) / dim));
623     const c1 = css("--c1"), ghost = css("--ghost"), muted = css("--muted"), surface = css("--surface");
624     const opts = baseOpts();
625     opts.interaction = { mode: "index", intersect: false };
626     opts.scales.x.ticks.maxTicksLimit = 10;
627     opts.scales.x.ticks.autoSkip = true;
628     opts.plugins.tooltip.callbacks = {
629         title: items => monthDate(month).toLocaleDateString("en-US", { month: "short" }) + " " + items[0].lab
630     },
631     el,
632     label: ctx => ctx.raw == null ? null : ` ${ctx.dataset.label}: ${money(ctx.raw)} `,
633     };
634     opts.plugins.tooltip.filter = item => item.raw != null;
635     makeChart("paceChart", {
636         type: "line",
637         data: {
638             labels: Array.from({ length: dim }, (_, i) => i + 1),
639             datasets: [
640                 { label: "This month", data: cur, borderColor: c1, backgroundColor: c1 + "22", fill: "origin", bo
641                 rderWidth: 2, tension: 0.25, pointRadius: cur.map((v, i) => i === lastDay - 1 ? 5 : 0), pointBackgroundColo
642                 r: c1, pointBorderColor: surface, pointBorderWidth: 2, pointHoverRadius: 5, order: 1 },
643                 { label: "Previous month", data: prev, borderColor: ghost, borderWidth: 2, tension: 0.25, pointRa
644                 dius: 0, pointHoverRadius: 4, fill: false, order: 2 },
645                 { label: "Budget pace", data: pace, borderColor: muted, borderDash: [5, 4], borderWidth: 1.5, poi
646                 ntRadius: 0, pointHoverRadius: 0, fill: false, order: 3 },
647             ],
648             options: opts,
649         });
650     }
651
652     function renderDow(list) {
653         const names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
654         const vals = Array(7).fill(0);
655         list.forEach(e => { const [y, m, d] = e.date.split("-").map(Number); vals[(new Date(y, m - 1, d).getDay
656         () + 6) % 7] += e.amount; });
657         const opts = baseOpts();
658         opts.scales.y.ticks.maxTicksLimit = 4;
659         opts.plugins.tooltip.callbacks = { label: ctx => ` ${money(ctx.raw)} ` };
660         makeChart("dowChart", { type: "bar", data: { labels: names, datasets: [{ label: "Spent", data: vals, ba
661         ckgroundColor: css("--c1"), borderRadius: { topLeft: 4, topRight: 4 }, borderSkipped: "bottom", maxBarThick
662         ness: 28 }] }, options: opts });
663     }
664
665     // ----- KPI / budgets / table -----
666     function renderKpis(list, cats, total) {
667         const prevTotal = sum(inMonth(shiftMonth(month, -1)));
668         $("kSpent").textContent = money(total);
669         if (prevTotal > 0) {
670             const d = (total - prevTotal) / prevTotal;
671             const isCurrent = month === ym(today);
672             $("kSpentSub").innerHTML = `<span class="delta ${d > 0 ? "up" : "down"} num">${d > 0 ? "▲" : "▼"} ${M
673             ath.abs(d * 100).toFixed(0)}%</span> vs ${money(prevTotal)} ${isCurrent ? "all of" : "in"} ${monthDate(shif
674             tMonth(month, -1)).toLocaleDateString("en-US", { month: "short" })}`;
675         } else $("kSpentSub").textContent = `${list.length} transactions`;
676
677         const tb = totalBudget(); const pct = tb ? total / tb : 0; const [cls, lbl] = status(pct);
678         $("kLeft").textContent = money(tb - total);
679         $("kMeter").style.width = Math.min(100, pct * 100).toFixed(1) + "%";
680         $("kMeter").style.background = `var(--${cls})`;
681         $("kLeftSub").innerHTML = `<span class="pill ${cls}">${lbl}</span><span class="num">${Math.round(pct *
682         100)}% of ${money(tb)}</span>`;
683
684         const isCurrent = month === ym(today);
685         const days = isCurrent ? today.getDate() : daysIn(month);
686         $("kDaily").textContent = money(days ? total / days : 0);
687         if (isCurrent) {
688             const proj = days ? total / days * daysIn(month) : 0;
689             $("kDailySub").innerHTML = `Projected month: <span class="num">${money(proj)}</span>`;
690         } else $("kDailySub").textContent = `Across ${days} days`;
691     }

```

```

681     const top = [...CATS].sort((a, b) => cats[b.id] - cats[a.id])[0];
682     if (total > 0) {
683         $("kTop").textContent = top.name;
684         $("kTopSub").innerHTML = `
692             <div class="row">
693                 <span class="name"><i class="swatch" style="--c:var(${c.v})"></i>${c.name} <span class="pill ${cl
        ↵ s}>${lbl}</span></span>
694                 <span class="fig">${money(s)} /
695                 <input type="number" min="0" step="100" id="bud-${c.id}" data-cat="${c.id}" value="${b}" aria-l
        ↵ abel="${c.name} monthly budget"></span>
696             </div>
697             <div class="bar${pct > 1 ? " over" : ""}" style="--c:var(${c.v})" role="img" aria-label="${c.name}:
        ↵ ${Math.round(pct * 100)}% of budget used"><span style="width:${Math.min(100, pct * 100).toFixed(1)}%"></sp
        ↵ an></div>
698         </li>`;
699     }).join("");
700     $("budgetList").querySelectorAll("input").forEach(inp => inp.addEventListener("change", e => {
701         const v = Math.max(0, parseFloat(e.target.value) || 0);
702         const cat = e.target.dataset.cat;
703         state.budgets[cat] = v; render();
704         persist(() => api("PUT", `budgets/${cat}`, { amount: v }));
705     }));
706 }
707
708 function renderCommitments(list, total) {
709     const emi = list.filter(e => e.cat === "emi"), sip = list.filter(e => e.cat === "sip");
710     const emiSum = sum(emi), sipSum = sum(sip);
711     const keys = Array.from({ length: 6 }, (_, i) => shiftMonth(month, i - 5));
712     const sip6 = keys.reduce((a, k) => a + sum(inMonth(k).filter(e => e.cat === "sip")), 0);
713     $("commitHint").textContent = monthLabel(month, "short");
714     $("emiTotal").textContent = money(emiSum);
715     $("emiSub").textContent = `${emi.length} payment${emi.length === 1 ? "" : "s"}`;
716     $("sipTotal").textContent = money(sipSum);
717     $("sipSub").textContent = `${sip.length} fund${sip.length === 1 ? "" : "s"}`;
718     $("commitNote").innerHTML = total
719     ? `EMI and SIP take <strong class="num">${Math.round((emiSum + sipSum) / total * 100)}%</strong> of t
        ↵ his month's outflow. SIPs over the last 6 months: <strong class="num">${money(sip6)}</strong>.`
720     : "No EMI or SIP payments this month.";
721     const items = [...emi, ...sip].sort((a, b) => a.date.localeCompare(b.date));
722     $("commitList").innerHTML = items.map(e => `- <i class="swatch" style="--c:var(${CAT[e.cat].v})"></i>
        ↵ <span>${esc(e.note)} <span class="d">· ${Number(e.date.slice(8, 10))} ${monthDate(month).toLocaleDateString
        ↵ ("en-US", { month: "short" })}</span></span><span class="a">${money(e.amount)}</span></li>`).join("");
723 }
724
725 function renderFilters() {
726     const chips = [{ id: "all", name: "All" }, ...CATS];
727     $("filters").innerHTML = chips.map(c => `<td>${ds}</td><td><span class="cat-tag"><i class="swatch" style="--c:var(${CAT[e.cat].v})
        ↵ "></i>${CAT[e.cat].name}</span></td><td class="note">${esc(e.note)}</td><td class="amt">${money(e.amount)}<
        ↵ /td><td><button class="del" data-id="${e.id}" aria-label="Delete ${esc(e.note)}" title="Delete">x</button><
        ↵ /td></tr>`;
738     }).join("") : `|<td colspan="5" class="empty">No expenses match. Add one with the form, or change th
        ↵ e filter.</td></tr>`;
739     $("tbody").querySelectorAll(".del").forEach(b => b.addEventListener("click", () => {
740         const id = b.dataset.id;
741         state.expenses = state.expenses.filter(e => e.id !== id); render();

|  |

```

```

742     persist(() => api("DELETE", `expenses/${encodeURIComponent(id)}`));
743   });
744   `${txCount}.textContent = `${list.length} transaction${list.length === 1 ? "" : "s"}`;
745   `${txSum}.textContent = `Total ${money(sum(list))}`;
746 }
747
748 function render() {
749   fillMonths();
750   $("#sampleNotice").hidden = !state.sample;
751   const list = inMonth(month), cats = byCat(list), total = sum(list);
752   renderKpis(list, cats, total);
753   renderTrend();
754   renderDonut(cats, total);
755   renderPace();
756   renderBudgets(cats);
757   renderCommitments(list, total);
758   renderDow(list);
759   renderFilters();
760   renderTable();
761 }
762
763 async function start() {
764   if (await detectServer()) {
765     try {
766       await loadFromServer();
767       serverMode = true;
768       $("#storeLabel").textContent = "Saved on your server";
769       $("#storeLabel").classList.add("server");
770       $("#storeText").textContent = "Changes are saved to your server.";
771       $("#csvLink").hidden = false;
772       $("#curSel").value = state.currency;
773     } catch (err) {
774       showSyncError(`Couldn't load your data from the server (${err.message}). Showing this browser's copy instead; changes won't reach the server until you reload.`);
775     }
776   }
777   setCurrency();
778   if (window.Chart) { Chart.defaults.font.family = "IBM Plex Sans, system-ui, sans-serif"; }
779   render();
780 }
781
782 // Re-render charts when the theme changes so they pick up the new tokens.
783 const rerender = () => { if (window.Chart) render(); };
784 try { matchMedia("(prefers-color-scheme: dark)").addEventListener("change", rerender); } catch (e) {}
785 new MutationObserver(rerender).observe(document.documentElement, { attributes: true, attributeFilter: ["data-theme"] });
786
787 if (document.fonts && document.fonts.ready) document.fonts.ready.then(() => { if (charts.trendChart) render(); });
788 start();
789 })();
790 </script>
791 <!-- ARTIFACT-BODY-END -->
792 </body>
793 </html>

```

backend/package.json: Dependencies and npm scripts

```
1 {
2   "name": "pocket-ledger-backend",
3   "version": "1.0.0",
4   "description": "REST API and SQLite storage for the Pocket Ledger expense dashboard",
5   "private": true,
6   "type": "module",
7   "main": "src/server.js",
8   "engines": {
9     "node": ">=22.13"
10  },
11  "scripts": {
12    "start": "node --disable-warning=ExperimentalWarning src/server.js",
13    "dev": "node --disable-warning=ExperimentalWarning --watch src/server.js",
14    "test": "node --disable-warning=ExperimentalWarning --test test/*.test.js"
15  },
16  "dependencies": {
17    "express": "^5.2.1"
18  }
19 }
```

backend/src/categories.js: Category ids, default budgets, currencies

```
1 // Expense categories. Ids must match the CATS list in ../index.html.
2 export const CATEGORIES = [
3   { id: "housing",    name: "Housing",      budget: 18000 },
4   { id: "food",       name: "Food & Dining", budget: 12000 },
5   { id: "transport",  name: "Transport",     budget: 6000 },
6   { id: "utilities",  name: "Utilities",     budget: 4500 },
7   { id: "shopping",   name: "Shopping",      budget: 6000 },
8   { id: "health",     name: "Health",        budget: 3000 },
9   { id: "fun",        name: "Entertainment", budget: 3500 },
10  { id: "other",       name: "Other",         budget: 2500 },
11  { id: "sip",         name: "SIP",           budget: 8000 },
12  { id: "emi",         name: "EMI",           budget: 12200 },
13 ];
14
15 export const CATEGORY_IDS = new Set(CATEGORIES.map(c => c.id));
16
17 export const CURRENCIES = new Set(["INR", "USD", "EUR", "GBP"]);
```

backend/src/db.js: SQLite schema and data access

```
1 import { DatabaseSync } from "node:sqlite";
2 import { mkdirSync } from "node:fs";
3 import { dirname } from "node:path";
4 import { CATEGORIES } from "../categories.js";
5
6 const SCHEMA = `
7   CREATE TABLE IF NOT EXISTS expenses (
8     id          TEXT PRIMARY KEY,
9     date        TEXT NOT NULL,
10    category    TEXT NOT NULL,
11    amount      REAL NOT NULL CHECK (amount > 0),
12    note        TEXT NOT NULL DEFAULT '',
13    created_at  TEXT NOT NULL DEFAULT (datetime('now'))
14  );
15  CREATE INDEX IF NOT EXISTS idx_expenses_date ON expenses (date);
16
17  CREATE TABLE IF NOT EXISTS budgets (
18    category TEXT PRIMARY KEY,
19    amount   REAL NOT NULL CHECK (amount >= 0)
20  );
21
22  CREATE TABLE IF NOT EXISTS settings (
23    key   TEXT PRIMARY KEY,
24    value TEXT NOT NULL
25  );
26 `;
27
28 // Opens (or creates) the database and returns a small data-access layer.
29 // Pass ":memory:" for a throwaway database (used by the tests).
30 export function openDb(path) {
31   if (path !== ":memory:") mkdirSync(dirname(path), { recursive: true });
32   const db = new DatabaseSync(path);
33   db.exec("PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON;");
34   db.exec(SCHEMA);
35
36   const insertBudget = db.prepare("INSERT OR IGNORE INTO budgets (category, amount) VALUES (?, ?)");
37   for (const c of CATEGORIES) insertBudget.run(c.id, c.budget);
38
39   const q = {
40     allExpenses: db.prepare("SELECT id, date, category, amount, note FROM expenses ORDER BY date DESC, amount DESC"),
41     monthExpenses: db.prepare("SELECT id, date, category, amount, note FROM expenses WHERE substr(date, 1, 7) = ? ORDER BY date DESC, amount DESC"),
42     getExpense: db.prepare("SELECT id, date, category, amount, note FROM expenses WHERE id = ?"),
43     insertExpense: db.prepare("INSERT INTO expenses (id, date, category, amount, note) VALUES (?, ?, ?, ?, ?)"),
44     updateExpense: db.prepare("UPDATE expenses SET date = ?, category = ?, amount = ?, note = ? WHERE id = ?"),
45     deleteExpense: db.prepare("DELETE FROM expenses WHERE id = ?"),
46     deleteAll: db.prepare("DELETE FROM expenses"),
47     budgets: db.prepare("SELECT category, amount FROM budgets"),
48     setBudget: db.prepare("INSERT INTO budgets (category, amount) VALUES (?, ?) ON CONFLICT (category) DO UPDATE SET amount = excluded.amount"),
49     settings: db.prepare("SELECT key, value FROM settings"),
50     setSetting: db.prepare("INSERT INTO settings (key, value) VALUES (?, ?) ON CONFLICT (key) DO UPDATE SET value = excluded.value"),
51     summary: db.prepare("SELECT category, SUM(amount) AS total, COUNT(*) AS count FROM expenses WHERE substr(date, 1, 7) = ? GROUP BY category"),
52   };
53
54   const tx = fn => {
55     db.exec("BEGIN");
56     try { const r = fn(); db.exec("COMMIT"); return r; }
57     catch (e) { db.exec("ROLLBACK"); throw e; }
58   };
59
60   return {
61     listExpenses: month => (month ? q.monthExpenses.all(month) : q.allExpenses.all()).map(plain),
62     getExpense: id => plain(q.getExpense.get(id)),
63     addExpense: e => { q.insertExpense.run(e.id, e.date, e.category, e.amount, e.note); return plain(q.getExpense.get(e.id)); },
64     updateExpense: (id, e) => q.updateExpense.run(e.date, e.category, e.amount, e.note, id).changes > 0,
65     deleteExpense: id => q.deleteExpense.run(id).changes > 0,
66     clearExpenses: () => tx(() => { q.deleteAll.run(); q.setSetting.run("sample", "false"); }),
67
68     budgets: () => Object.fromEntries(q.budgets.all().map(r => [r.category, r.amount])),
69     setBudget: (category, amount) => q.setBudget.run(category, amount),
70   };
71 }
```

```

70
71 settings: () => Object.fromEntries(q.settings.all().map(r => [r.key, r.value])),
72 setSetting: (key, value) => q.setSetting.run(key, String(value)),
73
74 summary: month => q.summary.all(month).map(plain),
75
76 // Loads a full ledger in one transaction (first-run sample data, or data moved from browser storage).
77 importLedger: ({ expenses, budgets, currency, sample, replace }) => tx(() => {
78   if (replace) q.deleteAll.run();
79   for (const e of expenses) q.insertExpense.run(e.id, e.date, e.category, e.amount, e.note);
80   for (const [c, v] of Object.entries(budgets || {})) q.setBudget.run(c, v);
81   if (currency) q.setSetting.run("currency", currency);
82   q.setSetting.run("sample", String(Boolean(sample)));
83   q.setSetting.run("initialized", "true");
84 },
85
86   close: () => db.close(),
87 };
88 }
89
90 // node:sqlite returns null-prototype rows; convert them to ordinary objects for JSON.
91 const plain = row => (row ? { ...row } : row);

```

backend/src/server.js: Express REST API, validation, login

```
1 import express from "express";
2 import { randomUUID, timingSafeEqual } from "node:crypto";
3 import { fileURLToPath } from "node:url";
4 import { dirname, join, resolve } from "node:path";
5 import { openDb } from "../db.js";
6 import { CATEGORIES, CATEGORY_IDS, CURRENCIES } from "../categories.js";
7
8 const HERE = dirname(fileURLToPath(import.meta.url));
9 const FRONTEND = resolve(HERE, "../../index.html");
10 const MAX_AMOUNT = 1e9;
11
12 class HttpError extends Error {
13   constructor(status, message) { super(message); this.status = status; }
14 }
15
16 // ----- Validation -----
17 function isRealDate(s) {
18   if (typeof s !== "string" || !/^\d{4}-\d{2}-\d{2}$/.test(s)) return false;
19   const [y, m, d] = s.split("-").map(Number);
20   const dt = new Date(Date.UTC(y, m - 1, d));
21   return dt.getUTCFullYear() === y && dt.getUTCMonth() === m - 1 && dt.getUTCDate() === d;
22 }
23
24 function parseExpense(body, { requireId = false } = {}) {
25   if (!body || typeof body !== "object") throw new HttpError(400, "Send the expense as a JSON object.");
26   const { id, date, category, amount, note = "" } = body;
27   if (!isRealDate(date)) throw new HttpError(400, "date must be a real calendar date in YYYY-MM-DD format.");
28   if (!CATEGORY_IDS.has(category)) throw new HttpError(400, `category must be one of: ${[...CATEGORY_IDS].join(", ")}`);
29   if (typeof amount !== "number" || !Number.isFinite(amount) || amount <= 0 || amount > MAX_AMOUNT)
30     throw new HttpError(400, "amount must be a number greater than 0.");
31   if (typeof note !== "string" || note.length > 200) throw new HttpError(400, "note must be text of at most 200 characters.");
32   if (id !== undefined && (typeof id !== "string" || !/^[A-Za-z0-9_-]{4,64}$/.test(id)))
33     throw new HttpError(400, "id may only contain letters, digits, '-' and '_' (4 to 64 characters).");
34   if (requireId && id === undefined) throw new HttpError(400, "id is required.");
35   return { id: id ?? randomUUID(), date, category, amount: Math.round(amount * 100) / 100, note: note.trim() };
36 }
37
38 function parseBudget(amount) {
39   if (typeof amount !== "number" || !Number.isFinite(amount) || amount < 0 || amount > MAX_AMOUNT)
40     throw new HttpError(400, "amount must be a number of 0 or more.");
41   return Math.round(amount * 100) / 100;
42 }
43
44 function parseMonth(month) {
45   if (month === undefined) return undefined;
46   if (typeof month !== "string" || !/^\d{4}-(0[1-9]|1[0-2])$/.test(month)) throw new HttpError(400, "month must be in YYYY-MM format.");
47   return month;
48 }
49
50 const csvCell = v => {
51   let s = String(v ?? "");
52   if (/^[=+@-@\t\r]$/i.test(s)) s = "'" + s; // stop spreadsheets from running cell text as a formula
53   return `["${s.replace(/"/g, '"')}"]`;
54 };
55
56 // Optional HTTP Basic auth: set APP_PASSWORD (and optionally APP_USER) to require a login.
57 function basicAuth(user, password) {
58   const expected = Buffer.from(`${user}:${password}`);
59   return (req, res, next) => {
60     const header = req.get("authorization") || "";
61     const given = Buffer.from(header.startsWith("Basic ") ? Buffer.from(header.slice(6), "base64").toString() : "");
62     if (given.length === expected.length && timingSafeEqual(given, expected)) return next();
63     res.set("WWW-Authenticate", `Basic realm="Pocket Ledger", charset="UTF-8"`).status(401).send("Login required.");
64   };
65 }
66
67 // ----- App -----
68 export function createApp(db, { password = process.env.APP_PASSWORD, user = process.env.APP_USER || "ledger" } = {}) {
69   const app = express();
```

```

70 app.disable("x-powered-by");
71 app.use((req, res, next) => { res.set("X-Content-Type-Options", "nosniff"); next(); });
72 if (password) app.use(basicAuth(user, password));
73 app.use(express.json({ limit: "5mb" }));
74
75 const api = express.Router();
76
77 api.get("/health", (req, res) => res.json({ ok: true }));
78
79 api.get("/categories", (req, res) => res.json(CATEGORIES));
80
81 // Everything the dashboard needs in one request.
82 api.get("/state", (req, res) => {
83   const s = db.settings();
84   res.json({
85     initialized: s.initialized === "true",
86     sample: s.sample === "true",
87     currency: s.currency || "INR",
88     budgets: db.budgets(),
89     expenses: db.listExpenses(),
90   });
91 });
92
93 api.get("/expenses", (req, res) => {
94   const month = parseMonth(req.query.month);
95   const category = req.query.category;
96   if (category !== undefined && !CATEGORY_IDS.has(category)) throw new HttpError(400, "Unknown category.");
97   let rows = db.listExpenses(month);
98   if (category) rows = rows.filter(r => r.category === category);
99   res.json(rows);
100 });
101
102 api.get("/expenses/:id", (req, res) => {
103   const row = db.getExpense(req.params.id);
104   if (!row) throw new HttpError(404, "No expense with that id.");
105   res.json(row);
106 });
107
108 api.post("/expenses", (req, res) => {
109   const e = parseExpense(req.body);
110   if (db.getExpense(e.id)) throw new HttpError(409, "An expense with that id already exists.");
111   res.status(201).json(db.addExpense(e));
112 });
113
114 api.put("/expenses/:id", (req, res) => {
115   const e = parseExpense({ ...req.body, id: req.params.id });
116   if (!db.updateExpense(req.params.id, e)) throw new HttpError(404, "No expense with that id.");
117   res.json(db.getExpense(req.params.id));
118 });
119
120 api.delete("/expenses/:id", (req, res) => {
121   if (!db.deleteExpense(req.params.id)) throw new HttpError(404, "No expense with that id.");
122   res.status(204).end();
123 });
124
125 // Removes every expense (used by "Clear sample data"). Budgets and settings are kept.
126 api.delete("/expenses", (req, res) => {
127   db.clearExpenses();
128   res.status(204).end();
129 });
130
131 api.get("/budgets", (req, res) => res.json(db.budgets()));
132
133 api.put("/budgets/:category", (req, res) => {
134   if (!CATEGORY_IDS.has(req.params.category)) throw new HttpError(404, "Unknown category.");
135   db.setBudget(req.params.category, parseBudget(req.body?.amount));
136   res.json(db.budgets());
137 });
138
139 api.put("/settings", (req, res) => {
140   const { currency } = req.body || {};
141   if (!CURRENCIES.has(currency)) throw new HttpError(400, `currency must be one of: ${[...CURRENCIES].join(", ")}`);
142   db.setSetting("currency", currency);
143   res.json({ currency });
144 });
145
146 // Month totals per category, for other clients or reports.

```

```

147 api.get("/summary", (req, res) => {
148   const month = parseMonth(req.query.month);
149   if (!month) throw new HttpError(400, "month is required, in YYYY-MM format.");
150   const rows = db.summary(month);
151   const budgets = db.budgets();
152   const total = rows.reduce((a, r) => a + r.total, 0);
153   const budgetTotal = Object.values(budgets).reduce((a, v) => a + v, 0);
154   res.json({
155     month, total, budgetTotal, remaining: budgetTotal - total,
156     categories: CATEGORIES.map(c => {
157       const r = rows.find(x => x.category === c.id);
158       return { id: c.id, name: c.name, spent: r?.total ?? 0, count: r?.count ?? 0, budget: budgets[c.id]
159         ?? 0 };
160     }),
161   });
162
163   // Loads a whole ledger at once: first-run sample data, or data moved over from browser storage.
164   // Refuses to overwrite an existing ledger unless "replace": true is sent.
165   api.post("/import", (req, res) => {
166     const { expenses, budgets = {}, currency, sample = false, replace = false } = req.body || {};
167     if (!Array.isArray(expenses) || expenses.length > 50000) throw new HttpError(400, "expenses must be an
168       array of at most 50,000 items.");
169     if (db.settings().initialized === "true" && !replace) throw new HttpError(409, "The ledger already has
170       data. Send \"replace\": true to overwrite it.");
171     const parsed = expenses.map(e => parseExpense(e));
172     if (new Set(parsed.map(e => e.id)).size !== parsed.length) throw new HttpError(400, "Expense ids must be
173       e unique.");
174     const cleanBudgets = {};
175     for (const [c, v] of Object.entries(budgets)) {
176       if (!CATEGORY_IDS.has(c)) throw new HttpError(400, `Unknown budget category: ${c}.`);
177       cleanBudgets[c] = parseBudget(v);
178     }
179     if (currency !== undefined && !CURRENCIES.has(currency)) throw new HttpError(400, "Unknown currency.");
180     db.importLedger({ expenses: parsed, budgets: cleanBudgets, currency, sample, replace });
181     res.status(201).json({ imported: parsed.length });
182   });
183
184   // Spreadsheet-friendly download of every expense.
185   api.get("/export.csv", (req, res) => {
186     const rows = db.listExpenses();
187     const name = Object.fromEntries(CATEGORIES.map(c => [c.id, c.name]));
188     const lines = ["date,category,amount,note", ...rows.map(r => [r.date, name[r.category] ?? r.category, r
189       .amount, r.note].map(csvCell).join(","))];
190     res.set("Content-Type", "text/csv; charset=utf-8");
191     res.set("Content-Disposition", "attachment; filename=pocket-ledger.csv");
192     res.send(lines.join("\n") + "\n");
193   });
194
195   app.use("/api", api);
196   app.use("/api", (req, res) => res.status(404).json({ error: "No such API endpoint." }));
197
198   // The dashboard itself.
199   app.get(["/", "/index.html"], (req, res) => res.sendFile(FRONTEND));
200
201   app.use((err, req, res, next) => {
202     const status = err.status || err.statusCode || 500;
203     if (status >= 500) console.error(err);
204     const message = err instanceof HttpError ? err.message
205       : err.type === "entity.parse.failed" ? "Request body is not valid JSON."
206       : err.type === "entity.too.large" ? "Request body is too large."
207       : "Something went wrong on the server.";
208     res.status(status).json({ error: message });
209   });
210
211   return app;
212 }
213
214 // ----- Start -----
215 if (process.argv[1] && resolve(process.argv[1]) === fileURLToPath(import.meta.url)) {
216   const port = Number(process.env.PORT) || 3000;
217   const dbPath = process.env.DB_PATH || join(HERE, "../data/ledger.db");
218   const db = openDb(dbPath);
219   const server = createApp(db).listen(port, () => {
220     console.log(`Pocket Ledger running at http://localhost:${port} (database: ${dbPath})`);
221     if (!process.env.APP_PASSWORD) console.log("Tip: set APP_PASSWORD to require a login before exposing th
222       is to the internet.");
223   });
224   const stop = () => server.close(() => { db.close(); process.exit(0); });

```

```
220 process.on("SIGINT", stop);
221 process.on("SIGTERM", stop);
222 }
```

backend/test/api.test.js: Automated API tests

```
1 import { test, before, after } from "node:test";
2 import assert from "node:assert/strict";
3 import { openDb } from "../src/db.js";
4 import { createApp } from "../src/server.js";
5
6 let server, base, db;
7
8 before(async () => {
9   db = openDb(":memory:");
10  server = createApp(db, { password: "" }).listen(0);
11  await new Promise(r => server.once("listening", r));
12  base = `http://127.0.0.1:${server.address().port}`;
13 });
14 after(() => { server.close(); db.close(); });
15
16 const call = async (method, path, body) => {
17   const res = await fetch(base + path, {
18     method,
19     headers: body !== undefined ? { "Content-Type": "application/json" } : {},
20     body: body !== undefined ? JSON.stringify(body) : undefined,
21   });
22   const text = await res.text();
23   let json; try { json = JSON.parse(text); } catch { json = text; }
24   return { status: res.status, body: json, headers: res.headers };
25 };
26
27 test("health check and dashboard page are served", async () => {
28   assert.deepEqual(await call("GET", "/api/health").body, { ok: true });
29   const page = await fetch(base + "/");
30   assert.equal(page.status, 200);
31   assert.match(await page.text(), /Pocket Ledger/);
32 });
33
34 test("a fresh database is uninitialized with default budgets", async () => {
35   const { body } = await call("GET", "/api/state");
36   assert.equal(body.initialized, false);
37   assert.equal(body.currency, "INR");
38   assert.equal(body.budgets.emi, 12200);
39   assert.equal(body.budgets.sip, 8000);
40   assert.deepEqual(body.expenses, []);
41 });
42
43 test("import loads a ledger once and refuses to overwrite without replace", async () => {
44   const ledger = {
45     expenses: [
46       { id: "seed0001", date: "2026-09-01", category: "housing", amount: 18000, note: "Rent" },
47       { id: "seed0002", date: "2026-09-05", category: "emi", amount: 9800, note: "Car loan EMI" },
48       { id: "seed0003", date: "2026-08-07", category: "sip", amount: 5000, note: "Index fund SIP" },
49     ],
50     budgets: { food: 10000 },
51     currency: "INR",
52     sample: true,
53   };
54   assert.equal(await call("POST", "/api/import", ledger).status, 201);
55   assert.equal(await call("POST", "/api/import", ledger).status, 409);
56
57   const { body } = await call("GET", "/api/state");
58   assert.equal(body.initialized, true);
59   assert.equal(body.sample, true);
60   assert.equal(body.expenses.length, 3);
61   assert.equal(body.budgets.food, 10000);
62 });
63
64 test("expenses can be added, read, updated, filtered and deleted", async () => {
65   const created = await call("POST", "/api/expenses", { id: "abc12345", date: "2026-09-10", category: "food",
66     ↪ amount: 450.456, note: "Lunch" });
67   assert.equal(created.status, 201);
68   assert.deepEqual(created.body, { id: "abc12345", date: "2026-09-10", category: "food", amount: 450.46, no
69     ↪ te: "Lunch" });
70
71   assert.equal(await call("POST", "/api/expenses", { id: "abc12345", date: "2026-09-10", category: "food",
72     ↪ amount: 1 })).status, 409);
73
74   const noId = await call("POST", "/api/expenses", { date: "2026-09-11", category: "transport", amount: 120
75     ↪ });
76   assert.equal(noId.status, 201);
77   assert.ok(noId.body.id.length >= 4);
78 }
```

```

74
75   const updated = await call("PUT", "/api/expenses/abc12345", { date: "2026-09-12", category: "food", amount: 500, note: "Team lunch" });
76   assert.equal(updated.body.amount, 500);
77   assert.equal(updated.body.note, "Team lunch");
78
79   const sept = await call("GET", "/api/expenses?month=2026-09&category=food");
80   assert.deepEqual(sept.body.map(e => e.id), ["abc12345"]);
81
82   assert.equal((await call("DELETE", "/api/expenses/abc12345")).status, 204);
83   assert.equal((await call("GET", "/api/expenses/abc12345")).status, 404);
84   assert.equal((await call("DELETE", "/api/expenses/abc12345")).status, 404);
85 });
86
87 test("invalid input is rejected with a helpful message", async () => {
88   const cases = [
89     { date: "2026-02-30", category: "food", amount: 10 },
90     { date: "2026-09-10", category: "crypto", amount: 10 },
91     { date: "2026-09-10", category: "food", amount: -5 },
92     { date: "2026-09-10", category: "food", amount: "10" },
93     { date: "2026-09-10", category: "food", amount: 10, id: "a b" },
94     { date: "2026-09-10", category: "food", amount: 10, note: "x".repeat(201) },
95   ];
96   for (const c of cases) {
97     const r = await call("POST", "/api/expenses", c);
98     assert.equal(r.status, 400, JSON.stringify(c));
99     assert.equal(typeof r.body.error, "string");
100   }
101   const bad = await fetch(base + "/api/expenses", { method: "POST", headers: { "Content-Type": "application/json" }, body: "{oops}" });
102   assert.equal(bad.status, 400);
103   assert.equal((await call("GET", "/api/expenses?month=2026-13")).status, 400);
104   assert.equal((await call("GET", "/api/nope")).status, 404);
105 });
106
107 test("budgets and currency can be changed", async () => {
108   const b = await call("PUT", "/api/budgets/emi", { amount: 15000 });
109   assert.equal(b.body.emi, 15000);
110   assert.equal((await call("PUT", "/api/budgets/emi", { amount: -1 })).status, 400);
111   assert.equal((await call("PUT", "/api/budgets/nothing", { amount: 1 })).status, 404);
112
113   assert.deepEqual((await call("PUT", "/api/settings", { currency: "USD" })).body, { currency: "USD" });
114   assert.equal((await call("PUT", "/api/settings", { currency: "BTC" })).status, 400);
115   assert.equal((await call("GET", "/api/state")).body.currency, "USD");
116 });
117
118 test("summary totals a month by category", async () => {
119   const { body } = await call("GET", "/api/summary?month=2026-09");
120   assert.equal(body.month, "2026-09");
121   const emi = body.categories.find(c => c.id === "emi");
122   assert.equal(emi.spent, 9800);
123   assert.equal(emi.budget, 15000);
124   assert.equal(body.total, body.categories.reduce((a, c) => a + c.spent, 0));
125   assert.equal((await call("GET", "/api/summary")).status, 400);
126 });
127
128 test("CSV export escapes text and neutralises formulas", async () => {
129   await call("POST", "/api/expenses", { id: "csv00001", date: "2026-09-20", category: "other", amount: 10, note: '=HYPERLINK("x"), "quoted"' });
130   const res = await fetch(base + "/api/export.csv");
131   assert.match(res.headers.get("content-type"), /text\/csv/);
132   const text = await res.text();
133   assert.match(text, /^date,category,amount,note\n/);
134   assert.ok(text.includes(`2026-09-20,other,10,"'=HYPERLINK("x"), "quoted"'"`));
135 });
136
137 test("clearing removes all expenses and turns off the sample flag", async () => {
138   assert.equal((await call("DELETE", "/api/expenses")).status, 204);
139   const { body } = await call("GET", "/api/state");
140   assert.deepEqual(body.expenses, []);
141   assert.equal(body.sample, false);
142   assert.equal(body.budgets.emi, 15000);
143 });
144
145 test("APP_PASSWORD turns on a login", async () => {
146   const locked = createApp(openDb(":memory:"), { user: "me", password: "s3cret" }).listen(0);
147   await new Promise(r => locked.once("listening", r));
148   const url = `http://127.0.0.1:${locked.address().port}/api/health`;
149   try {

```

```
150     assert.equal(await fetch(url).status, 401);
151     const wrong = await fetch(url, { headers: { Authorization: "Basic " + Buffer.from("me:nope").toString("
↳ base64") } });
152     assert.equal(wrong.status, 401);
153     const ok = await fetch(url, { headers: { Authorization: "Basic " + Buffer.from("me:s3cret").toString("b
↳ ase64") } });
154     assert.equal(ok.status, 200);
155   } finally { locked.close(); }
156 });
```

Dockerfile: Container image for deployment

```
1 FROM node:22-alpine
2 WORKDIR /app
3 COPY backend/package.json backend/package-lock.json ./backend/
4 RUN cd backend && npm ci --omit=dev
5 COPY index.html ./
6 COPY backend/src ./backend/src
7 ENV NODE_ENV=production PORT=3000 DB_PATH=/data/ledger.db
8 RUN mkdir -p /data && chown node:node /data
9 VOLUME /data
10 EXPOSE 3000
11 USER node
12 CMD ["node", "--disable-warning=ExperimentalWarning", "backend/src/server.js"]
```